

☒ Easy Level (10 Questions)

Q1. What is a class?

- A. An instance of an object
- B. A blueprint for creating objects
- C. A variable
- D. A function

Answer: B

Explanation: A class defines structure and behavior.

Q2. What keyword is used in Python to define a class?

- A. function
- B. object
- C. class
- D. def

Answer: C

Explanation: class declares a class.

Q3. What is an object?

- A. A class
- B. An instance of a class
- C. A function

D. A method

Answer: B

Explanation: Object is an instance of a class.

Q4. What method initializes an object in Python?

A. `__start__`

B. `__init__`

C. `initialize()`

D. `start()`

Answer: B

Explanation: `__init__` is the constructor method.

Q5. Which OOP concept hides data?

A. Inheritance

B. Polymorphism

C. Encapsulation

D. Abstraction

Answer: C

Explanation: Encapsulation restricts direct data access.

Q6. What is inheritance?

A. One class inherits from another

- B. Combining two classes
- C. Copying objects
- D. Defining functions

Answer: A

Explanation: Child class inherits attributes and methods.

Q7. What does self refer to in Python class methods?

- A. The class itself
- B. The current instance of the class
- C. A global variable
- D. The parent class

Answer: B

Explanation: self points to the current object.

Q8. Which method allows child classes to have different behavior from parent?

- A. Overloading
- B. Overriding
- C. Encapsulation
- D. Abstraction

Answer: B

Explanation: Method overriding changes behavior.

Q9. Linux kernel uses which language mainly?

- A. Python
- B. Java
- C. C
- D. Ruby

Answer: C

Explanation: Linux kernel is primarily written in C.

Q10. Which module in Python supports abstract classes?

- A. abc
- B. abstract
- C. os
- D. sys

Answer: A

Explanation: abc module supports abstraction.

☒ Medium Level (30 Questions)

Q11. What happens when a child class calls `super().__init__()`?

- A. It calls the parent class constructor
- B. It calls child's own constructor
- C. Raises error

D. Calls unrelated function

Answer: A

Explanation: super() accesses parent class methods.

Q12. What is polymorphism in OOP?

A. Classes with many methods

B. Ability to use same interface for different data types

C. Multiple inheritance

D. Encapsulation

Answer: B

Explanation: Polymorphism allows same method name with different behaviors.

Q13. What is the output?

python

CopyEdit

```
class A: def method(self): return "A" class B(A): def method(self): return "B" obj = B() print(obj.method())
```

- A. A
- B. B
- C. Error
- D. None

Answer: B

Explanation: Child overrides parent method.

Q14. Which of these is NOT a valid OOP principle?

- A. Encapsulation
- B. Compilation
- C. Inheritance
- D. Abstraction

Answer: B

Explanation: Compilation is not an OOP principle.

Q15. In Linux, what file extension do shared libraries typically have?

- A. .dll
- B. .so
- C. .lib
- D. .exe

Answer: B

Explanation: Linux shared libraries end with .so.

Q16. Which of these keywords is used to inherit a class in Python?

- A. extends
- B. inherits
- C. Parent class name in parentheses after child class name
- D. implements

Answer: C

Explanation: Example: class Child(Parent):

Q17. How do you make a variable private in Python (convention)?

- A. Prefix with __ (double underscore)
- B. Prefix with _ (single underscore)
- C. Use private keyword
- D. Use private_var keyword

Answer: A

Explanation: Double underscore invokes name mangling.

Q18. What is method overloading?

- A. Defining multiple methods with same name but different parameters
- B. Defining methods with different names
- C. Calling parent method
- D. None

Answer: A

Explanation: Overloading means same method name, different signatures.

Q19. Does Python support method overloading natively?

A. Yes

B. No

Answer: B

Explanation: Python does not support overloading directly.

Q20. Which of the following is a mutable data type?

A. Tuple

B. String

C. List

D. Int

Answer: C

Explanation: Lists can be modified after creation.

Q21. How do you define a class attribute (shared by all instances)?

A. Inside methods

B. Outside methods but inside class

C. Inside `__init__` only

D. Outside class

Answer: B

Explanation: Class attributes defined at class level.

Q22. What is the output?

python

CopyEdit

```
class Test: count = 0
def __init__(self): Test.count += 1
a = Test()
b = Test()
print(Test.count)
```

- A. 0
- B. 1
- C. 2
- D. Error

Answer: C

Explanation: Class attribute count increments on each instantiation.

Q23. What is the difference between a class method and instance method?

- A. Class methods use @classmethod decorator and take cls parameter
- B. Instance methods take self parameter
- C. Both A and B
- D. None

Answer: C

Explanation: Class methods operate on class, instance methods on objects.

Q24. Which of these is an example of multiple inheritance?

- A. class C(A, B):
- B. class C(A):
- C. class C:
- D. None

Answer: A

Explanation: Inheriting from multiple classes.

Q25. Which of the following is NOT a feature of OOP?

- A. Encapsulation
- B. Polymorphism
- C. Procedural code
- D. Abstraction

Answer: C

Explanation: Procedural code is non-OOP.

Q26. What does the term “instance variable” mean?

- A. Variable shared across all instances
- B. Variable unique to each object
- C. Global variable

D. None

Answer: B

Explanation: Instance variables belong to objects.

Q27. How do you access a parent class method inside a child class?

A. parent.method()

B. super().method()

C. self.method()

D. parent::method()

Answer: B

Explanation: super() calls parent methods.

Q28. What is encapsulation's benefit?

A. Data hiding and protection

B. Faster code

C. Less memory usage

D. More errors

Answer: A

Explanation: Protects internal data.

Q29. What does this code print?

python

CopyEdit

```
class A: def __init__(self): self.x = 5 a = A() print(a.x)
```

- A. 5
- B. None
- C. Error
- D. 0

Answer: A

Explanation: Prints instance variable x.

Q30. Which of the following best describes abstraction?

- A. Hiding complex details and showing only necessary features
- B. Combining data and methods
- C. Creating objects
- D. Accessing data directly

Answer: A

Explanation: Abstraction hides complexity.

☒ Hard Level (10 Questions)

Q31. What is the method resolution order (MRO) in Python?

- A. The order Python uses to search methods in inheritance hierarchy
- B. Order of method calls in a program
- C. Order of method definitions
- D. None

Answer: A

Explanation: MRO defines search order in multiple inheritance.

Q32. What is the output?

python

CopyEdit

```
class A: def foo(self): print("A") class B(A): def foo(self): print("B") class C(A): def foo(self): print("C") class D(B, C): pass d = D() d.foo()
```

- A. A
- B. B
- C. C
- D. Error

Answer: B

Explanation: Python follows MRO, B's method is called first.

Q33. How do you declare an abstract class in Python?

- A. Using @abstractclass decorator
- B. Inheriting from abc.ABC and using @abstractmethod
- C. Using abstract keyword
- D. Using interface keyword

Answer: B

Explanation: Abstract classes use abc module.

Q34. What is a metaclass in Python?

- A. A class that defines behavior of classes themselves
- B. A regular class
- C. A method in class
- D. None

Answer: A

Explanation: Metaclasses control class creation.

Q35. Which method is called when an object is deleted?

- A. `__del__`
- B. `__destroy__`
- C. `delete()`
- D. `remove()`

Answer: A

Explanation: `__del__` is the destructor method.

Q36. What will happen if you try to instantiate an abstract class?

- A. Error
- B. Object created
- C. Warning
- D. None

Answer: A

Explanation: Abstract classes cannot be instantiated.

Q37. What does the diamond problem refer to?

- A. Ambiguity in multiple inheritance
- B. Problem with diamond shapes
- C. Syntax error
- D. Runtime error

Answer: A

Explanation: Multiple inheritance ambiguity.

Q38. What is the difference between shallow and deep copy?

- A. Shallow copies references; deep copies objects recursively
- B. No difference
- C. Shallow copies objects; deep copies references
- D. None

Answer: A

Explanation: Deep copy duplicates nested objects.

Q39. How is encapsulation enforced in Python?

- A. Name mangling with double underscore prefix
- B. Using private keyword
- C. Using protected keyword
- D. Python does not support encapsulation

Answer: A

Explanation: Uses name mangling to limit access.

Q40. Which of these statements about Python's super() is false?

- A. It calls the immediate parent method
- B. It can be used in multiple inheritance
- C. It can call grandparent methods
- D. It cannot be used inside class methods

Answer: D

Explanation: super() can be used inside class methods.
